

Database Schema and SQL Installation File

Project: RoomRadar QC

RoomRadar QC uses a MySQL database to store semester information, room information, and class schedule records. The backend also includes Python logic that creates the required tables automatically when the Flask application starts, but the SQL statements are provided here so the database can be manually reviewed, created, and installed if needed.

1. Database Overview

RoomRadar QC stores course schedule data so users can search for rooms that appear available during a selected day and time range.

The database contains three main tables:

Table	Purpose
<code>semesters</code>	Stores uploaded semester names.
<code>rooms</code>	Stores room information such as room code, building code, and capacity.
<code>schedules</code>	Stores class schedule records connected to rooms and semesters.

2. Backend Database Structure

The backend database connection is handled through the `get_db()` function. This function connects to MySQL using the database configuration stored in `DB_CONFIG`.

The backend table creation is handled by the `create_tables()` function. This function creates the required tables if they do not already exist.

The Flask application calls `create_tables()` when the backend starts. This allows the project to initialize the required database tables automatically during local development.

3. Schema Description

3.1 semesters Table

The `semesters` table stores the name of each uploaded semester.

Column	Data Type	Description
<code>id</code>	<code>INT AUTO_INCREMENT PRIMARY KEY</code>	Unique identifier for each semester.
<code>name</code>	<code>VARCHAR(50)</code>	Name of the uploaded semester, such as <code>Fall 2026</code> .

3.2 rooms Table

The `rooms` table stores room records from the uploaded schedule data.

Column	Data Type	Description
<code>id</code>	<code>INT AUTO_INCREMENT PRIMARY KEY</code>	Unique identifier for each room.
<code>room_code</code>	<code>VARCHAR(20)</code>	Full room code, such as <code>SB E243</code> or <code>PH 212</code> .
<code>building</code>	<code>VARCHAR(100)</code>	Building code, such as <code>SB</code> , <code>PH</code> , <code>KY</code> , or <code>RE</code> .
<code>capacity</code>	<code>INT</code>	Room capacity from the uploaded schedule data.

3.3 schedules Table

The `schedules` table stores class meeting records.

Column	Data Type	Description
<code>id</code>	<code>INT AUTO INCREMENT PRIMARY KEY</code>	Unique identifier for each schedule record.
<code>room_id</code>	<code>INT</code>	References the room where the class takes place.

Column	Data Type	Description
<code>semester_id</code>	INT	References the semester that the schedule belongs to.
<code>course_name</code>	VARCHAR(100)	Course name or course description.
<code>instructor</code>	VARCHAR(100)	Instructor name.
<code>day</code>	VARCHAR(20)	Day value from the uploaded CSV, such as <code>M</code> , <code>T</code> , <code>TH</code> , <code>M,W</code> , <code>S</code> , or <code>SU</code> .
<code>start_time</code>	TIME	Class start time.
<code>end_time</code>	TIME	Class end time.

4. Table Relationships

The database uses foreign key relationships in the `schedules` table.

Relationship	Description
<code>schedules.room_id → rooms.id</code>	Each schedule record belongs to one room.
<code>schedules.semester_id → semesters.id</code>	Each schedule record belongs to one semester.

Relationship summary:

```
semesters.id  -----< schedules.semester_id
rooms.id      -----< schedules.room_id
```

This means:

- One semester can have many schedule records.
- One room can have many schedule records.
- Each schedule record belongs to one room.
- Each schedule record belongs to one semester.

5. SQL Installation Script

The following SQL script creates the database and all required tables.

Save this as:

```
install_database.sql
```

```
-- RoomRadar QC Database Installation Script
-- Database: MySQL
-- Purpose: Create the RoomRadar QC database and required tables.
```

```
CREATE DATABASE IF NOT EXISTS roomradar_qc;
```

```
USE roomradar_qc;
```

```
CREATE TABLE IF NOT EXISTS semesters (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50)
);
```

```
CREATE TABLE IF NOT EXISTS rooms (
    id INT AUTO_INCREMENT PRIMARY KEY,
    room_code VARCHAR(20),
    building VARCHAR(100),
    capacity INT
);
```

```
CREATE TABLE IF NOT EXISTS schedules (
    id INT AUTO_INCREMENT PRIMARY KEY,
    room_id INT,
    semester_id INT,
    course_name VARCHAR(100),
    instructor VARCHAR(100),
    day VARCHAR(20),
    start_time TIME,
    end_time TIME,
    FOREIGN KEY (room_id) REFERENCES rooms(id),
    FOREIGN KEY (semester_id) REFERENCES semesters(id)
);
```

6. SQL Table Creation Script

If the database already exists, use this script to create only the required tables.

Save this as:

```
create_tables.sql

-- RoomRadar QC Table Creation Script
-- Database: MySQL
-- Purpose: Create the required RoomRadar QC tables.

CREATE TABLE IF NOT EXISTS semesters (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS rooms (
    id INT AUTO_INCREMENT PRIMARY KEY,
    room_code VARCHAR(20),
    building VARCHAR(100),
    capacity INT
);

CREATE TABLE IF NOT EXISTS schedules (
    id INT AUTO_INCREMENT PRIMARY KEY,
    room_id INT,
    semester_id INT,
    course_name VARCHAR(100),
    instructor VARCHAR(100),
    day VARCHAR(20),
    start_time TIME,
    end_time TIME,
    FOREIGN KEY (room_id) REFERENCES rooms(id),
    FOREIGN KEY (semester_id) REFERENCES semesters(id)
);
```

7. SQL Cleanup Script

This script can be used if the database tables need to be dropped and recreated.

Save this as:

```
drop_tables.sql
```

```
-- RoomRadar QC Cleanup Script  
-- Warning: This deletes the project tables and their data.
```

```
SET FOREIGN_KEY_CHECKS = 0;
```

```
DROP TABLE IF EXISTS schedules;  
DROP TABLE IF EXISTS rooms;  
DROP TABLE IF EXISTS semesters;
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

8. Optional Sample Data Script

This script inserts a small amount of sample data for testing. It is not the full Queens College semester schedule.

Save this as:

```
sample_data.sql
```

```
-- RoomRadar QC Sample Data  
-- Purpose: Optional test data for verifying the database setup.
```

```
INSERT INTO semesters (name)  
VALUES ('Sample Semester');
```

```
INSERT INTO rooms (room_code, building, capacity)  
VALUES  
    ('SB E243', 'SB', 30),  
    ('PH 212', 'PH', 35),  
    ('KY 242', 'KY', 40);
```

```
INSERT INTO schedules (  
    room_id,  
    semester_id,  
    course_name,  
    instructor,
```

```
    day,  
    start_time,  
    end_time  
)  
VALUES  
  (1, 1, 'Sample Environmental Science Class', 'Sample Instructor', 'T',  
   (2, 1, 'Sample Education Class', 'Sample Instructor', 'M,W', '10:00:00',  
   (3, 1, 'Sample Computer Science Class', 'Sample Instructor', 'F', '09:00:00',
```

9. Installation Instructions

Option 1: Full Database Setup

Use this option if the `roomradar_qc` database does not already exist.

1. Open MySQL.
2. Run the `install_database.sql` file.
3. Confirm that the `semesters`, `rooms`, and `schedules` tables were created.
4. Update the backend `DB_CONFIG` settings so the Flask backend connects to the correct database.

Example command inside MySQL:

```
SOURCE install_database.sql;
```

Option 2: Create Tables Only

Use this option if the database already exists.

```
USE roomradar_qc;  
SOURCE create_tables.sql;
```

Option 3: Insert Sample Data

After the tables are created, optional sample data can be inserted with:

```
SOURCE sample_data.sql;
```

Option 4: Clean Reinstall

Use this option only if the database needs to be reset.

```
SOURCE drop_tables.sql;
SOURCE create_tables.sql;
SOURCE sample_data.sql;
```

Warning: `drop_tables.sql` removes the existing project tables and their data.

10. How the Backend Uses the Database

10.1 Application Startup

When the Flask backend starts, it registers the room and admin routes and calls the table creation function.

This means the backend can automatically create the required tables if they do not already exist.

10.2 Admin Semester Upload

The admin upload route allows an authenticated admin user to upload semester schedule data.

The upload process:

1. Checks for an `Authorization` header.
2. Verifies the Firebase ID token.
3. Reads the semester name from the form.
4. Reads the uploaded CSV file.
5. Inserts a new semester into the `semesters` table.
6. Loops through each CSV row.
7. Skips non-in-person classes.
8. Skips empty or online locations.

9. Checks whether the room already exists.
 10. Inserts the room if it does not already exist.
 11. Inserts the class schedule into the `schedules` table.
-

10.3 Room Search

The room search endpoint receives:

- Day
- Start time
- End time
- Optional building code

The backend returns rooms that do not have a scheduled class conflict during the selected time range.

A room is considered unavailable if the scheduled class overlaps with the selected search range.

Overlap logic:

```
scheduled_start_time < requested_end_time
AND
scheduled_end_time > requested_start_time
```

Example SQL logic:

```
SELECT DISTINCT rooms.id, rooms.room_code, rooms.building
FROM rooms
WHERE rooms.id NOT IN (
    SELECT room_id
    FROM schedules
    WHERE day = 'T'
    AND start_time < '15:30:00'
    AND end_time > '12:30:00'
);
```

This query returns rooms that are not scheduled during the selected Tuesday time range.

10.4 Room Schedule Details

The room details endpoint returns the schedule for a selected room and day.

Example SQL logic:

```
SELECT course_name, instructor, day, start_time, end_time
FROM schedules
WHERE room_id = 1
AND day = 'T'
ORDER BY start_time;
```

This allows the frontend to display the selected room’s schedule.

11. CSV Fields Used by the Database

The admin upload process expects the uploaded CSV file to contain certain fields.

CSV Field	Database Usage
Mode of Instruction	Used to include only in-person classes.
Location	Used to determine the room code and building code.
Limit	Used as room capacity.
Time	Used to create start_time and end_time .
Description	Stored as course_name .
Instructor	Stored as instructor .
Day	Stored as the schedule day.

12. Day Format

The schedule data uses short day codes.

Day Code	Meaning
M	Monday
T	Tuesday
W	Wednesday
TH	Thursday

Day Code	Meaning
F	Friday
S	Saturday
SU	Sunday
M, W	Monday and Wednesday
T, TH	Tuesday and Thursday
W, F	Wednesday and Friday

The frontend can use full day names, and the backend maps them to the matching CSV day codes.

Frontend Day	Backend Day Code
Monday	M
Tuesday	T
Wednesday	W
Thursday	TH
Friday	F
Saturday	S
Sunday	SU

13. Notes and Limitations

- The database depends on accurate uploaded semester schedule data.
- The system uses scheduled class times, not real-time occupancy.
- The system cannot determine whether a room is locked or physically occupied.
- The day values must match the expected CSV day format.
- The schema is designed for Queens College schedule data and building codes.
- The schema can be expanded in the future to support more room details, stronger admin roles, multiple campuses, or room feature filters.